MEMORY DESIGN AND DATA HANDLING NOTE

BSE4104 EMERGING TRENDS IN SOFTWARE ENGINEERING

University Student-Support Case Agent

Group M Evening - Week 6

1. Purpose and Scope

This note defines the one justified persistent-memory use case for the University Student-Support Case Agent, as required by the Week 6 brief. It documents what is stored, why, who can access it, and how it is retained and deleted. The Agent Task Contract (Section 4) already named this gap directly: “Persistent memory across browser sessions is out of scope for Week 5; planned for Week 6 per project charter.” This note is that plan.

Scope is deliberately narrow: one use case, not a general memory subsystem. The Week 6 brief lists case history as one of its own examples of a justified use case, and it is the cleanest fit for this project, since the system already writes a durable, structured case record every time create_support_case succeeds (data/cases/drafts.json, built in Week 4). This note does not propose a new store. It proposes a careful, bounded read path over a store that already exists and is already tested.

This note is a specification, not an implementation. The actual code (“Working memory/state demonstration”) is a separate Week 6 deliverable owned by a teammate, and the State Model (also Week 6) defines the session-state baseline this note builds on, including the Layer A (turn), Layer B (session) and Layer C (draft store) terms used below. Every interface in Section 10 is written to be implemented directly, the same way the Week 4 Tool Catalogue specified retrieve_documents and create_support_case precisely enough for teammates to build against without further clarification.

2. The One Persistent-Memory Use Case: Case History Recall

When a student returns in a new chat session, whether that is a new browser tab, a reload, or simply a different day, the agent currently has no memory of any case drafted for them earlier. SessionState (src/agent/state.py, Week 5; Layer B in the State Model) tracks active_case_id, but only for the lifetime of that one session, and it is cleared when the session ends. A student who already has an open case, and who comes back and describes the same issue again, gets no acknowledgement that a case already exists. The only way for them to find out is to ask a staff member directly.

Case History Recall closes that gap. When a student in a new session identifies themselves with their own registration number, and their message suggests they already have an existing case (“I already reported this”, “what happened to my case”, or simply restating an issue matching one on file), the orchestration layer looks up any existing case drafts for that exact registration number and gives the model that information as context. The reply can then say “you already have case CASE-2026-000123 open for this” instead of silently offering to create a duplicate.

It is worth saying why this use case was chosen over the brief's other examples. An “approved preference” has no natural analogue in this domain, since the agent does not personalise onboarding content, and inventing a preference store would be memory without a justified task behind it. A “prior task result” is, in this project, exactly what a case record already is, so Case History Recall is really that same idea applied to the one durable artefact this system already produces.

3. What Is Stored

No new fields and no new store. The memory is data/cases/drafts.json, unchanged from Week 4, reused as a second-purpose read source rather than duplicated. In the State Model's terms this is Layer C, the durable draft store; Layers A and B are not changed by this feature. Every field already has a justification from its original design in case_store.py; this table restates that justification from the memory use case's point of view.

Field	Type	Why it is kept as memory
case_id	string	The stable handle a returning student's case is recalled by, and what the agent names back to them.
session_id	string	Ties the record to the session that created it, for audit. It is not used to recognise a returning session, since Week 5 sessions are not persisted and a new session always gets a new session_id.
student_identifier	string	The only lookup key. The student supplies it in this message or an earlier turn of the same conversation. No new identity mechanism is introduced.
issue_summary	string	Lets the agent confirm a recalled case is about the same issue the student is now describing, rather than assuming any case under that identifier is the relevant one.
category	string	Tells the agent (and the student) which office is already handling this, without re-deriving it.
queue_label	string	Same as category; lets the agent state where the case sits without guessing.
created_at	string (ISO 8601)	Lets the agent say how long ago the case was opened, and lets a future retention job identify old records (Section 8).

4. Why No New Tool Is Added

The Agent Task Contract, Section 2, is explicit: “The agent has exactly two tools.” A third, model-callable tool for case lookup was considered and rejected for that reason. Adding one would mean either breaking that line of the Contract or forcing a renegotiation of it mid-project, for a capability that does not actually need model-driven tool-calling to work.

Case History Recall is instead implemented as a deterministic Sense-phase step in src/tools/orchestrate.py, the same mechanism already used for the Week 5 “session already has a draft” note. Application code decides, from state and the message alone, whether relevant case history exists, and if so appends a short, factual note to the system prompt before the model is ever called. The model never requests this information and never decides whether to look it up. It only reads what deterministic code has already placed in front of it, the same way it already reads retrieved document chunks.

This keeps the tool count at two, keeps the Boundary Matrix's deterministic/AI split intact (lookup and filtering are deterministic code; only the wording of the reply is left to the model), and avoids opening a new authorization surface that Week 4's Failure/Authorization Test Evidence never exercised.

5. Retrieval Design

Trigger (Sense phase, before the model is called): both conditions below must hold.

- The number matches REGISTRATION_RE in case_store.py. It may be in this message or an earlier turn. If no number is present, no lookup runs.
- The current message matches a narrow "prior case" phrase, such as "already reported", "my case", "what happened to", "still waiting", or "follow up on". This is the same deliberately narrow, explicit-statement style already used for the Week 5 non-freshman check (S4), not an attempt to infer intent from arbitrary phrasing.

If state.active_case_id is already set for the current session, this step is skipped entirely, since the agent already knows about that case from Week 5's within-session memory and a second lookup would be redundant.

Lookup: a new deterministic function, exact-match only, on the existing store.

```
def get_case_history(student_identifier: str) -> list[dict]:
    """Read-only. Exact match on student_identifier only - no partial
    or fuzzy matching, so a lookup can never surface another
    student's record through a near-miss identifier."""
    records = _load()  # same helper already in tools/case_store.py
    identifier = student_identifier.strip().upper()
    return [r for r in records if r["student_identifier"] == identifier]
```

Injection: if any records are found, a short note is appended to the system prompt, following the same pattern already used for the per-session draft cap.

> Session note: 23/U/22615 has 1 case, CASE-2026-000123,
> queue ict_helpdesk, opened 2026-09-30.
> Issue: cannot activate webmail.
> If this is the same issue, say that case already exists.
> The application refuses a second draft for this number.

If no records are found, or the phrase is absent, nothing is injected and the turn proceeds as it does today. If drafts.json already has a case for that registration number, the application refuses create_support_case for a second case.

6. Access Control

Who / what	Access level	Basis
The model (Gemini)	Read-only, and only the records matching the exact identifier the student typed in this conversation	Receives only the injected system-prompt note; never calls get_case_history itself, since it is not a declared tool
Orchestration layer (src/tools/orchestrate.py)	Full read of get_case_history's result for the current turn	Needed to build the injected note; same trust level the Week 4/5 code already has over tools/case_store.py
Support-desk staff	Full read of data/cases/drafts.json	Unchanged from Week 4; staff were always the intended reader of this file
Other students	Can hear a summary	A typed number can reveal that student's case summary. No list or browse view. See Section 9.
External systems	Gemini prompt	Sent to Gemini inside the prompt.

		No office receives the drafts file.

7. Why This Does Not Silently Control Critical Decisions

Case History Recall is read-only: it cannot create, modify, resolve, or close a case. It can only tell the model that a case already exists, in plain text, for the model to relay to the student. The decision to look anything up at all is made by application code, not the model, so the agent can never choose to run a lookup it hasn't been given, and it never decides on its own that a student "counts" as a returning case-holder. Identity is exact-match only and never inferred: the student types their own registration number, exactly as they already do when creating a case in the first place, so this feature introduces no new way for the agent to guess who it is talking to. The known limitation of that trust model is not new and is not hidden; it is carried forward and discussed in full in Section 9.

The agreement gate, the blocked-topic check, and the fee-decision check stay as they are.
One rule is added. If drafts.json already has a case for that registration number,
the application refuses create_support_case. The model does not make that decision.
If the lookup finds nothing, or the trigger never fires, the turn proceeds exactly
as it does today. Admissions, grading, discipline, and fee decisions stay with staff.

8. Retention and Deletion Policy

Item	Policy
What is retained	Case drafts already written by create_support_case (Week 4). No new data is captured for this feature.
Retention period during the capstone	Indefinite for the 8-week build. The project uses only synthetic, team-created student records (Project Charter, Assumptions and Constraints), so no real student is affected by this records' retention, and an immediate deletion mechanism is not a safety requirement for this phase.
Deletion mechanism (proposed, not yet built)	A manual, staff-run script (the State Model's "manual retention policy" for Layer C) that removes drafts.json records older than a configurable window, keyed on created_at, the field that already exists for this reason (Section 3). This is deliberately not exposed to the model or to students: deletion is a staff/admin action, matching the Boundary Matrix's principle that irreversible or destructive actions stay outside the agent's authority.
Before any real deployment	This retention policy is explicitly a capstone-phase placeholder. A production deployment with real students would need: a defined retention period tied to an actual university records policy, a verified-identity deletion request path (not a typed registration number, per the limitation in Section 9), and a decision on whether resolved cases are deleted or archived. None of this is in scope for Week 6; it is named here so it is not silently forgotten.

9. Known Limitations and Risks

The main risk is identity. Exactly like create_support_case and the Week 5 non-freshman check, a student's registration number is trusted as typed and is not verified against any authentication system, so someone who knows or guesses another student's registration number could see that student's case summary. This is an accepted limitation for a capstone build on synthetic data, carried forward from

Week 4's design rather than introduced by this feature, but it has to be fixed before any deployment on real student data.

The phrasing heuristic that triggers a lookup will miss some genuine follow-ups and occasionally fire when it shouldn't. Like every other deterministic Sense-phase check in this project, blocked_topic, fee_decision, and the non-freshman regex included, the “prior case” phrasing list is kept narrow on purpose, favouring a missed recall over the agent confidently stating something that isn't true. Finally, if a student has more than one case on file, the injected note lists all of them, and the agent still has to use issue_summary to judge which one, if any, matches the current message. The one-draft-per-session cap from Week 5 should keep this rare within a single session, but it can still happen across several sessions over time.

10. Interface Specification for Implementation

For whoever builds “Working memory/state demonstration”: this is the exact surface to implement against.

10.1 New function: tools/case_store.py

```
def get_case_history(student_identifier: str) -> list[dict]:
    """See Section 5. Exact match only. Returns [] if none found
    or if student_identifier is empty/malformed - never raises for
    a not-found case, mirroring create_draft's existing style."""
```

10.2 New Sense-phase step: tools/orchestrate.py

```
# After the existing Sense checks (blocked_topic, fee_decision,
# non-freshman, override attempt) and before load_prompt("v1.3"):

identifier = _extract_registration_number(state)  # this message, then earlier turns
    if identifier and _looks_like_prior_case_reference(question):

if history:
    system_prompt += _format_case_history_note(identifier, history)
    refuse_new_draft(identifier)
```

10.3 State model coordination

No change to SessionState or to the State Model is required. Layers A and B stay exactly as the State Model defines them. The feature adds one read path from Layer C into the system prompt for the current turn, and the recalled records live only in a local variable for that turn. They are not stored on SessionState, so the State Model's list of session fields stays complete as written. Because the lookup is a small local file read, there is nothing worth caching between turns.